Homework — 2.2.1 Programming Techniques — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [2] — 1 mark: **count-controlled** iteration repeats a **fixed number of times, known in advance**; **condition-controlled** iteration repeats until/while a **condition** holds, so the number of repeats is not known in advance. 1 mark: correct constructs — count-controlled: `for ... next`; condition-controlled: `while ... endwhile` or `do ... until`.

2. [2] — 1 mark: `do ... until` tests its condition at the **end** of the loop, so the body always runs **at least once**. 1 mark: the input must be read at least once before it can be tested, so the at-least-once behaviour fits validation exactly; a `while` loop tests first, so it would need the value to exist before the loop / would need duplicated input code before the loop.

3. [3] — Trace of `power(2, 4)` (calls descend, then returns unwind):

```
power(2,4) = 2 * power(2,3)
power(2,3) = 2 * power(2,2)
power(2,2) = 2 * power(2,1)
power(2,1) = 2 * power(2,0)
power(2,0) = 1            (base case, exp == 0)
```

Unwinding: 2×1 = 2 → 2×2 = 4 → 2×4 = 8 → 2×8 = **16**. - 1 mark: base case identified — `power(2,0)` returns 1. - 1 mark: correct returned values at each level (2, 4, 8, 16) shown on the unwind. - 1 mark: final result **16**.

4. - (a) **[1]** — Output line 1: **0**; Output line 2: **8** (both required for the mark). - (b) **[2]** — 1 mark: the assignment inside `reset()` creates a **local** variable `count`, whose scope is the procedure only, so line 1 prints 0. 1 mark: the **global** `count` is a separate variable and is never changed by the procedure, so after the call line 2 prints 8.

5. [2] — 1 mark: after `scaleByVal(total)`, **total = 10** — passing **by value** gives the procedure a **copy**, so the original variable is **unchanged**. 1 mark: after `scaleByRef(total)`, **total = 20** — passing **by reference** passes the variable's **memory address**, so assigning to `n` changes the original. *(The consequence — unchanged / changed to 20 — must be stated; "a copy is made" alone is not enough.)*

6. [2] — 1 mark: **encapsulation** is combining the attributes (data) and methods that act on them into a class, keeping the attributes **private** so they can only be accessed/changed via the class's public methods. 1 mark for one benefit: protects data integrity / prevents other code changing attributes to invalid values / changes to the internal implementation do not affect code that uses the class / easier maintenance.

Part B (6 marks)

7. [2] — 1 mark: **abstraction** is removing/hiding unnecessary detail to focus on what matters for the problem. 1 mark (any one): makes the problem simpler/more manageable; reduces complexity; produces a general model that can be reused; lets the solver ignore irrelevant detail.

8. [2] — 1 mark: stack = **LIFO** (Last In, First Out). 1 mark: queue = **FIFO** (First In, First Out).

9. [2] — 1 mark: **decomposition** is breaking a complex problem down into smaller sub-problems. 1 mark: each sub-problem is simpler to solve/test separately (and the work can be divided among a team or reused).

Total: 14 + 6 = 20 marks